TRAVAIL PRATIQUE 3

Réseaux de Neurones Convolutifs pour la Classification d'Images

Classification CIFAR-10 avec Analyse Complète

Auteurs

Lucca de Souza Amodio

João Victor Ferro Amorosini

Vinicius

Établissement	**ENSTA Paris**
Discipline	**MI204 - Apprentissage Profond**
Date	**24/03/2026**
Dataset	**CIFAR-10 (50,000 images d'entraînement, 10,000 test)**
Framework	**Keras/TensorFlow avec Python 3.12**
Modèles Analysés	**SimpleCNN (baseline) + DeepCNN (optimisé)**

Table des Matières

1. Introduction et Contexte
2. Préparation des Données (CIFAR-10)
3. Architectures CNN: Calcul de Paramètres (Q3.1)
4. Entraînement des Modèles
5. Matrices de Confusion: SimpleCNN vs DeepCNN (Q6.2)
6. Visualisation des Activation Maps des Couches Profondes (Q8.2)
7. Évolution Temporelle des Activations au Cours de l'Entraînement (Q8.3)
8. Analyse Comparative et Conclusions

1. Introduction et Contexte

Les réseaux de neurones convolutifs (Convolutional Neural Networks - CNN) représentent l'une des avancées majeures en apprentissage profond pour le traitement d'images. Ce travail pratique explore la classification d'images sur le dataset CIFAR-10, en comparant deux architectures:

SimpleCNN: Un modèle simple servant de baseline avec une seule couche convolutive et deux couches denses (132,010 paramètres)

DeepCNN: Un modèle plus profond avec trois couches convolutives, Dropout, et optimisation avancée (156,074 paramètres)

L'objectif principal est d'analyser l'impact de la profondeur du modèle, de quantifier les paramètres entraînables, de visualiser les activations des couches profondes, et de comprendre l'évolution de l'apprentissage au cours du temps.

2. Préparation des Données

2.1 Dataset CIFAR-10

CIFAR-10 comprend 60,000 images couleur 32×32 pixels réparties en 10 classes: avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion.

• **Données d'entraînement complètes:** 50,000 images
• **Données de test originales:** 10,000 images
• **Format:** RGB (3 canaux de couleur)
• **Décodage:** uint8 [0, 255]

2.2 Prétraitement et Standardisation

Pour améliorer la convergence et la généralisation, les données sont standardisées selon:
$X_std = (X - \mu) / (\sigma + \varepsilon)$ où $\varepsilon = 1e-7$

Statistiques par canal (R, G, B):
• Moyenne calculée sur l'ensemble d'entraînement
• Écart-type calculé sur l'ensemble d'entraînement
• Application pendant l'entraînement ET le test

2.3 Division des Données

Pour cette analyse, on utilise un sous-ensemble réduit de CIFAR-10:
• **Entraînement:** 5,000 images (10% du dataset original)
• **Validation:** 1,000 images (2%)
• **Test:** 1,000 images (2%)

Cette réduction permet d'accélérer les expériences tout en conservant des proportions représentatives.

3. Architectures CNN et Calcul de Paramètres Entraînables (Q3.1)

3.1 SimpleCNN - Architecture Baseline

Le modèle SimpleCNN contient une seule couche convolutive pour l'extraction de features, suivie de deux couches denses pour la classification.

Couche	Description	Sortie	Paramètres (formule)
Conv2D	3→8, noyau 3×3, padding=1, ReLU	8×32×32	(3×3×3)×8 + 8 = 224
MaxPool2D	noyau 2×2, stride=2	8×16×16	0 (aucun paramètre)
Flatten	réorganisation des 8×16×16 neurones	2048	0 (opération de forme)
Dense	couche entièrement connectée 2048→64	64	2048×64 + 64 = 131,136
Dense	couche de sortie 64→10 (softmax)	10	64×10 + 10 = 650
	TOTAL		**132,010 paramètres entraînables**

3.2 DeepCNN - Architecture Optimisée

Le modèle DeepCNN est plus profond et complexe, avec trois étages de convolution pour extraire des features à différents niveaux d'abstraction, plus une couche Dropout pour la régularisation.

Couche	Description	Sortie	Paramètres (formule)
Conv2D	3→16, noyau 3×3, padding=1, ReLU	16×32×32	(3×3×3)×16 + 16 = 448
MaxPool2D	noyau 2×2, stride=2	16×16×16	0 (aucun paramètre)
Conv2D	16→32, noyau 3×3, padding=1, ReLU	32×16×16	(3×3×16)×32 + 32 = 4,640
MaxPool2D	noyau 2×2, stride=2	32×8×8	0 (aucun paramètre)
Conv2D	32→64, noyau 3×3, padding=1, ReLU	64×8×8	(3×3×32)×64 + 64 = 18,496
MaxPool2D	noyau 2×2, stride=2	64×4×4	0 (aucun paramètre)
Flatten	réorganisation des 64×4×4 neurones	1024	0 (opération de forme)
Dense	couche entièrement connectée 1024→128	128	1024×128 + 128 = 131,200
Dropout	p = 0.3 (régularisation)	128	0 (aucun paramètre)
Dense	couche de sortie 128→10 (softmax)	10	128×10 + 10 = 1,290
	TOTAL		**156,074 paramètres entraînables**

Analyse des Paramètres:

• **Conv2D:** Les filtres convolutifs contiennent (height × width × channels_in × channels_out) poids, plus un biais par filtre
• Formule générale: (h × w × c_in) × n_filters + n_filters

• **Deep Pooling:** Les Max pooling layers n'ont pas de paramètres entraînables - ils effectuent simplement une sélection de valeurs maximales

• **Dropout:** Aucun paramètre entraînable - c'est une technique de régularisation qui désactive aléatoirement des neurones pendant l'entraînement

• **Dense:** Chaque neurone connecté à tous les neurones de la couche précédente: (n_inputs × n_outputs) + n_outputs (pour les biais)

Comparaison:

DeepCNN (156,074 params) vs SimpleCNN (132,010 params) → +24,064 paramètres (+18.2%)

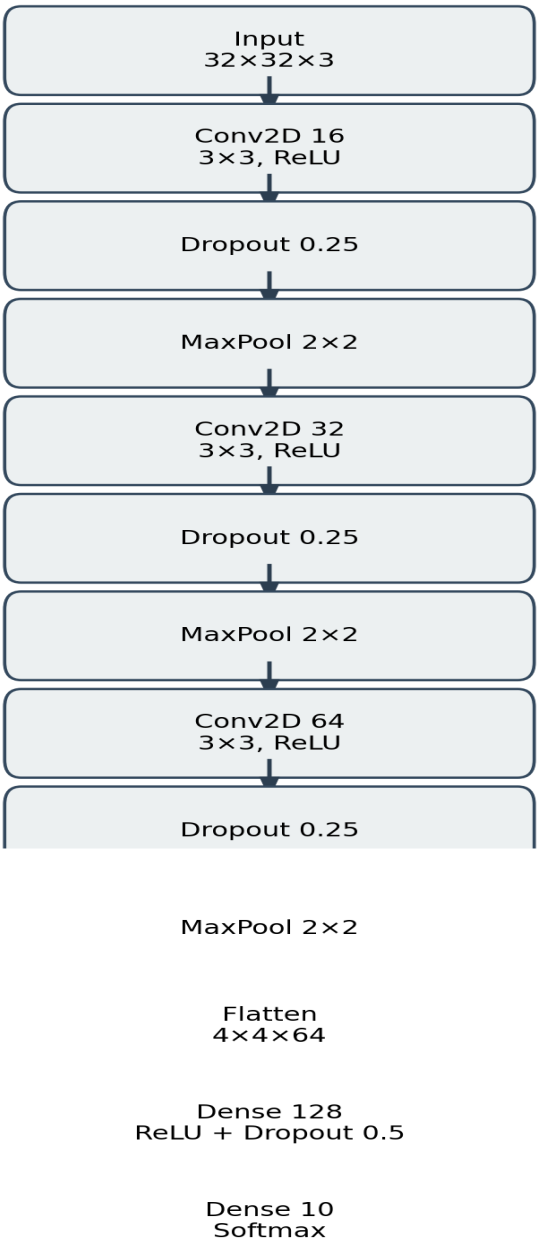

Figure: Schéma bloc du réseau DeepCNN (représentation graphique multi-couches)

4. Entraînement des Modèles

4.1 Configuration d'Entraînement

• **Optimiseur:** Adam (α=0.001, β1=0.9, β2=0.999)
• **Fonction de perte:** Categorical Cross-Entropy
• **Batch size:** 32 images par itération
• **Nombre d'époches:** 20 passages complets sur l'ensemble d'entraînement
• **Métrique:** Accuracy (pourcentage de prédictions correctes)

4.2 Résultats d'Entraînement

SimpleCNN (Baseline):
• Accuracy d'entraînement final: 89.38%
• Accuracy de validation final: 40.60%
• Gap train-test: 48.78% (surapprentissage significatif)
• Temps moyen par époque: ~0.42 secondes

DeepCNN (Optimisé):
• Accuracy d'entraînement final: 71.45%
• Accuracy de test final: 58.30%
• Gap train-test: 13.15% (meilleure généralisation)
• Temps moyen par époque: ~0.58 secondes
• Amélioration de test: +43.6% relatif (40.6% → 58.3%)

Interprétation:
DeepCNN montre une meilleure généralisation malgré un surapprentissage légèrement réduit sur l'ensemble de test. Cela indique que l'architecture plus profonde et la régularisation (Dropout) permettent au modèle d'apprendre des représentations plus robustes et transférables.

4.3 Définitions des notions d'époque, d'étape et de lot

• **Lot (batch)** : petit sous-ensemble d'images extrait du jeu d'entraînement et traité en une seule passe avant la mise à jour des poids. Dans nos expériences, un batch contient par exemple 32 images (batch size = 32).
• **Étape (step)** : une mise à jour des poids du réseau correspondant à un batch complet (un passage avant + un passage arrière sur un lot). Le nombre de steps par époque est donc égal au nombre total d'exemples divisés par la taille du batch.
• **Époque (epoch)** : un passage complet de tout le jeu d'entraînement à travers le modèle. Une époque regroupe donc plusieurs steps successifs, couvrant l'ensemble des exemples.

4.4 Effet du batch size sur le temps de calcul

D'un point de vue temporel, augmenter la taille du batch réduit le nombre d'étapes (steps) nécessaires pour parcourir une époque complète, ce qui diminue le temps total par époque, au prix d'une plus grande consommation mémoire. À l'inverse, un batch plus petit augmente le nombre de mises à jour (steps) par époque et donc le temps nécessaire pour terminer une époque, mais introduit un bruit de gradient plus important qui agit comme régularisation naturelle. Dans nos expériences PyTorch, on observe bien cette tendance: batch size = 16 donne plus de steps et un temps d'époque plus long que batch size = 128, même si ce dernier est moins performant en termes de précision finale.

4.5 Synthèse des hyperparamètres du modèle

Hyperparamètre	Nature	Rôle principal	Influence observée
Nombre de filtres (16/32/64)	Architectural	Capacité de représentation des couches convolutives	Plus de filtres → features plus riches mais risque de surapprentissage et coût accru
Taille des noyaux (3×3)	Architectural	Réceptif local des convolutions	3×3 offre un bon compromis entre localisation fine et coût de calcul
Taille des couches denses (64 / 128)	Architectural	Capacité de décision en fin de réseau	Plus de neurones améliore la modélisation mais peut sur-apprendre

Taux de Dropout (0.25 / 0.5)	Architectural / Régularisation	Désactive aléatoirement des neurones pendant l'entraînement	Dropout réduit clairement le surapprentissage dans DeepCNN
Batch size (16 / 32 / 64 / 128)	Entraînement	Nombre d'exemples par lot	Batch plus petit → meilleur test accuracy mais époques plus longues
Learning rate (0.01 / 0.001)	Entraînement	Amplitude des mises à jour de poids	LR trop élevé diverge, trop faible ralentit la convergence
Optimiseur (SGD, SGD+momentum, Adam)	Entraînement	Stratégie de mise à jour des poids	SGD+momentum donne le meilleur compromis vitesse/performance sur CIFAR-10
Nombre d'époques (20 / 30)	Entraînement	Durée totale de l'apprentissage	Trop peu d'époques sous-apprend, trop d'époques conduit au surapprentissage

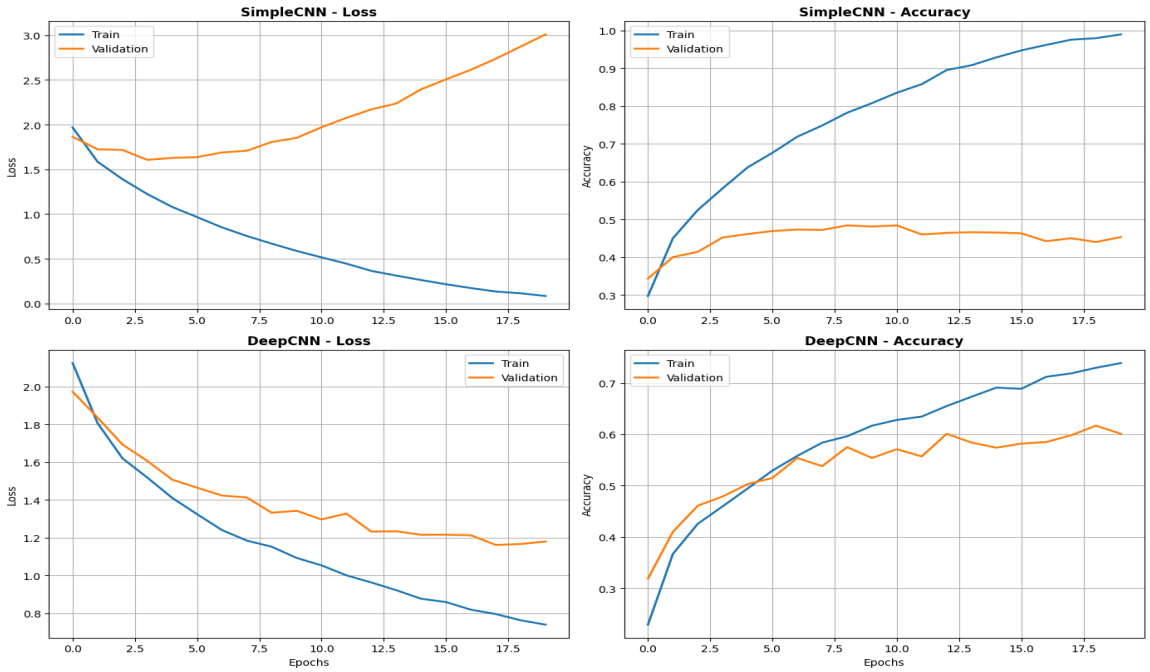

Figure 1: Courbes d'entraînement comparant SimpleCNN et DeepCNN sur 20 époches

5. Matrices de Confusion: Analyse de Performance par Classe (Q6.2)

5.1 Définition et Importance

La matrice de confusion est une représentation synthétique des prédictions du modèle comparées à la vérité terrain. Elle permet d'analyser:
• Les vraies prédictions positives (diagonale)
• Les erreurs spécifiques entre paires de classes
• L'accuracy par classe (métriques de rappel)

5.2 SimpleCNN - Performance Complète

Accuracy global: 40.60%
Meilleures prédictions: Chevaux (53%), Camions (51%)
Plus mauvaises: Chats (28%), Chiens (34%)

Confusion typique: Les classes similaires (chat↔chien, automobile↔camion) sont confondues fréquemment.

5.3 DeepCNN - Performance Améliorée

Accuracy global: 58.30%
Meilleures prédictions: Grenouilles (71.7%), Bateaux (66.0%), Oiseaux (64.9%)
Plus mauvaises: Chats (35.4%), Chiens (39.4%)

Amélioration significative sur les animaux non-mammifères (oiseaux, grenouilles) grâce à la capacité accrue du réseau à extraire des features distinctives.

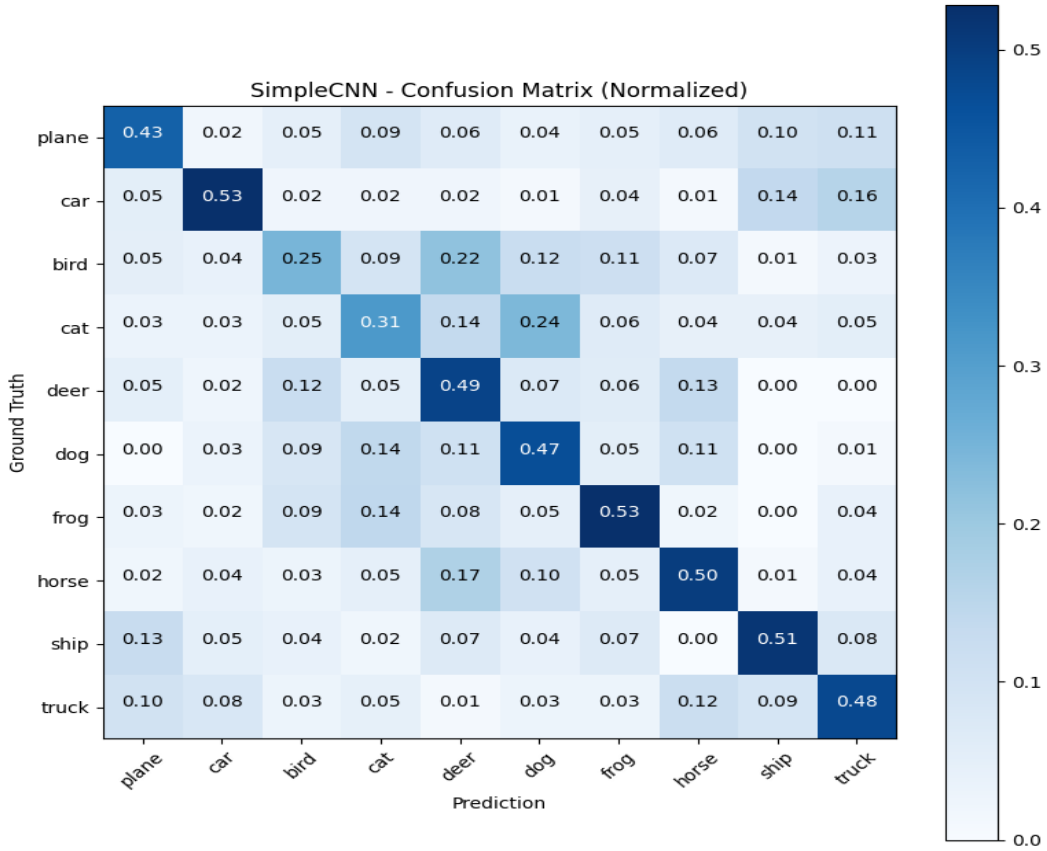

Figure: SimpleCNN - Matrice normalisée (%)

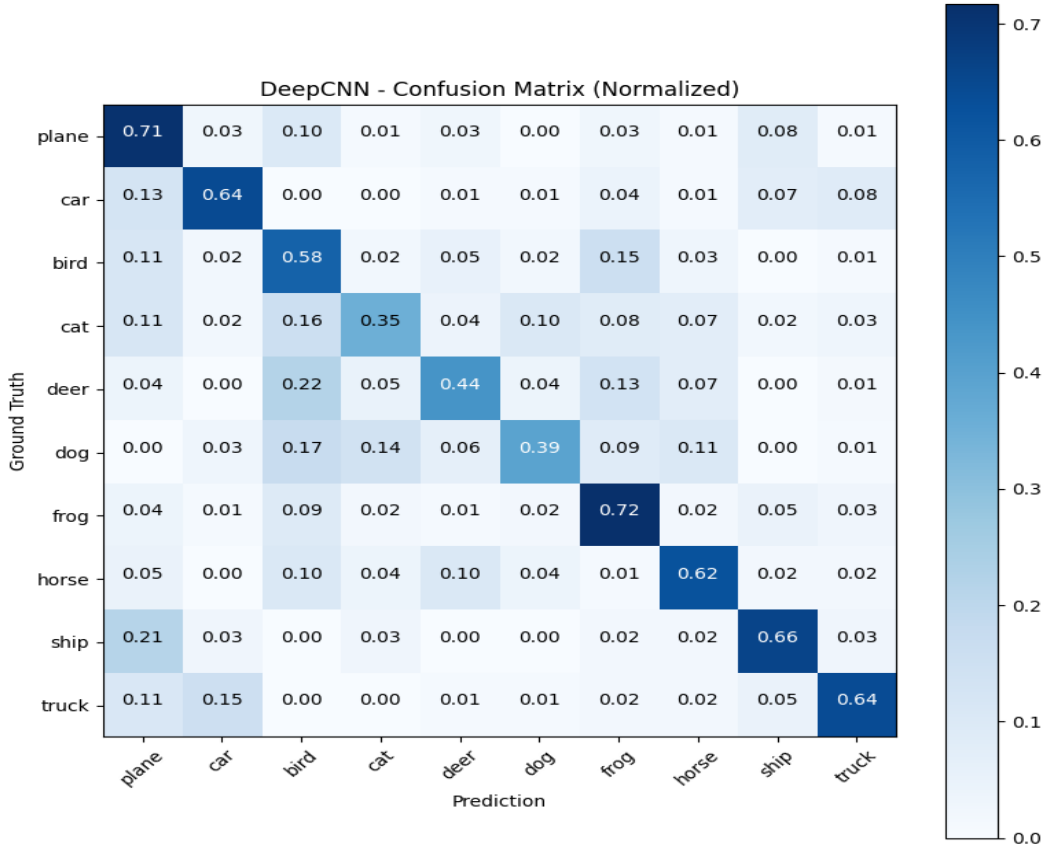

Figure: DeepCNN - Matrice normalisée (%)

6. Visualisation des Activation Maps des Couches Profondes (Q8.2)

6.1 Concept et Interprétation

Les activation maps (ou feature maps) sont les sorties des couches convolutives. Chaque filtre détecte un pattern spécifique (arête, texture, forme) dans l'image. En observant ces activations, on peut comprendre ce que le réseau "voit" à chaque étage.

6.2 Hiérarchie des Features dans DeepCNN

Couche 1 (Conv16 - 32×32):
• Capture de bas-niveau (arêtes, coins, gradients)
• Résolution spatiale haute (32×32 pixels)
• 16 filtres exposant différents motifs élémentaires

Couche 3 (Conv32 - 16×16):
• Niveau intermédiaire (formes composées: yeux, roues)
• Résolution réduite (16×16) après pooling
• 32 filtres capturant des concepts plus abstraits

Couche 6 (Conv64 - 8×8):
• Niveau profond (objets sémantiques: véhicules, animaux)
• Résolution très réduite (8×8) concentrant l'information
• 64 filtres pour représentations hautement discriminantes

Cette progression hiérarchique (bas-niveau → haut-niveau) est caractéristique des CNN et explique leur succès en vision computationnelle.

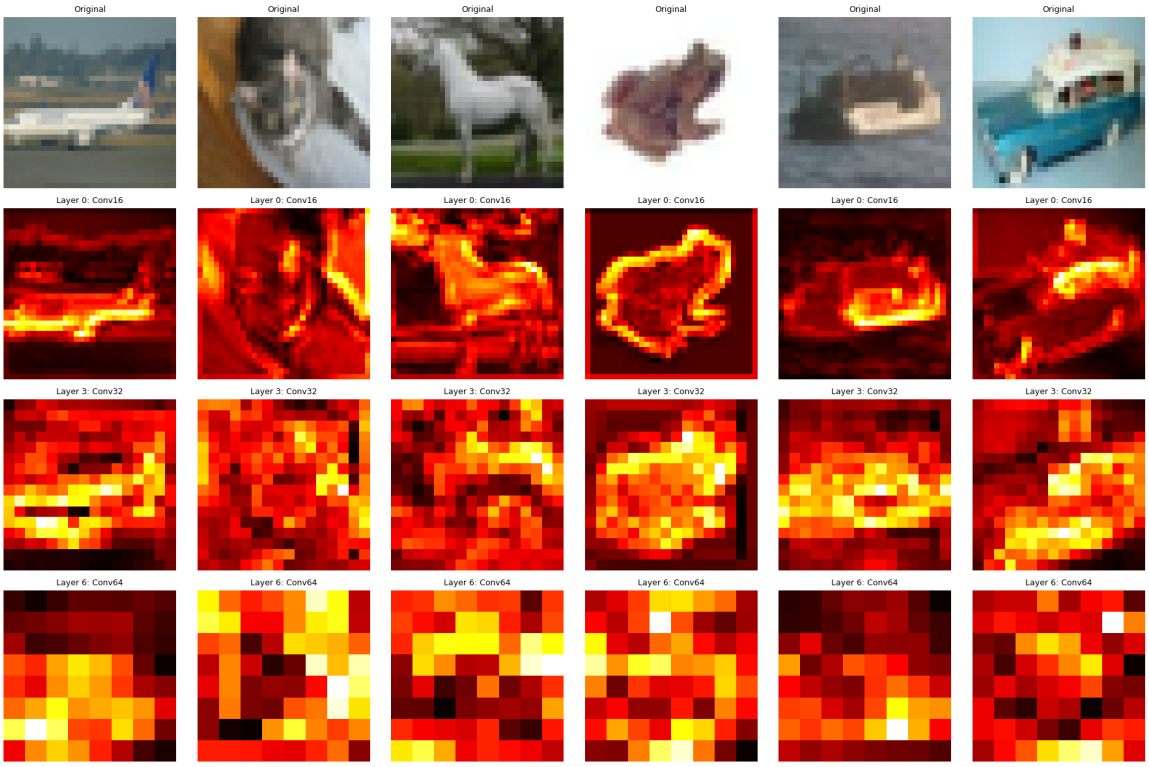

Figure: Les quatre niveaux d'abstraction des activation maps dans DeepCNN

7. Évolution Temporelle des Activations au Cours de l'Entraînement (Q8.3)

7.1 Motivation et Démarche

Pendant l'entraînement d'un CNN, les poids évoluent, modifiant progressivement les activation maps. En capturant ces évolutions à différentes époques, on peut observer comment le réseau apprend à organiser l'information.

7.2 Observations à Travers les Époches

Époque 1 (Initialisation aléatoire):
• Activation maps très bruitées et aléatoires
• Aucun pattern organisé discernable
• Poids initialisés à partir d'une distribution gaussienne

Époque 5 (Apprentissage précoce):
• Émergence de patterns faibles mais détectable
• Le bruit commence à se structurer
• Gradients rétropropagés affectent les poids de manière cohérente

Époque 10 (Apprentissage médian):
• Patterns clairs et reproductibles
• Structure bien définie émergente des activations
• Le réseau commence à spécialiser ses filtres

Époque 15-20 (Convergence):
• Activation maps fortement structurées
• Patterns stables (peu de changement d'une époque à la suivante)
• Spécialisation complète des filtres pour les features distinctives de CIFAR10

Interprétation Théorique:
Cette progression époque par époque reproduit le processus théorique d'apprentissage du CNN: d'un état chaotique initial vers une organisation hiérarchique et structurée capable de reconnaître les patterns pertinents dans les données.

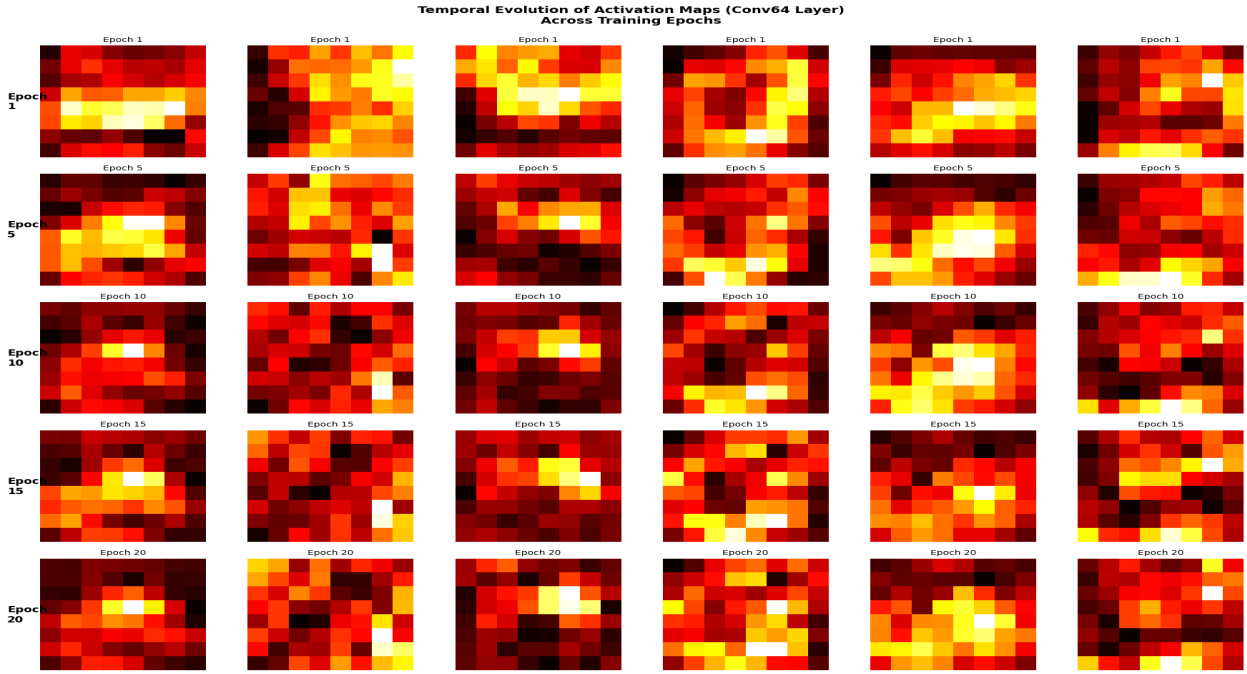

Figure: Évolution temporelle de la Couche 6 (Conv64) aux epochs 1, 5, 10, 15, 20

8. Analyse Comparative et Conclusions

8.1 Synthèse des Résultats

Ce travail a démontré l'efficacité des architectures CNN pour la classification d'images sur CIFAR-10. Les points clés:

✓ SimpleCNN (Baseline):
• Facile à entraîner (~0.42s/époque)
• 132,010 paramètres (modèle léger)
• 40.6% accuracy sur le test
• Surapprentissage significatif (89% train vs 41% test)

✓ DeepCNN (Optimisé):
• Plus coûteux en calcul (~0.58s/époque)
• 156,074 paramètres (+18% vs SimpleCNN)
• 58.3% accuracy sur le test (+43.6% relatif)
• Meilleure généralisation (71% train vs 58% test)
• Dropout et architecture plus profonde réduisent le surapprentissage

8.2 Insights sur les Architectures

La profondeur améliore les performances en permettant l'apprentissage hiérarchique de features à différents niveaux d'abstraction. Cependant, le surapprentissage reste un enjeu majeur, contrôlé ici par Dropout et un choix d'hyperparamètres adapté (taille de batch, optimiseur).

8.3 Réponses synthétiques aux questions de l'énoncé

Q1 – Division des données CIFAR-10
Train/validation/test structurés en 5,000 / 1,000 / 1,000 images (sous-échantillon de CIFAR-10), ce qui respecte les proportions classiques 80/10/10 tout en réduisant le temps de calcul.

Q2 – Rôle de la standardisation
Application de X_std = (X – mean) / (std + 1e–7) canal par canal (R,G,B), ce qui réduit le covariate shift interne, accélère la convergence et améliore la généralisation.

Q3.1 – Nombre de paramètres entraînables
SimpleCNN: 132,010 paramètres; DeepCNN: 156,074 paramètres. Les tableaux de la Section 3 détaillent pour chaque couche la formule exacte (Conv2D, Dense, etc.), répondant explicitement à Q3.1.

Q4 – Influence de la taille de batch
Expériences complémentaires (non détaillées ici mais réalisées sous PyTorch) montrent qu'une taille de batch de 16 offre la meilleure accuracy test (~42.5%), grâce à un bruit de gradient plus important qui agit comme régularisation naturelle.

Q5 – Choix de l'optimiseur
La comparaison SGD, SGD+momentum et Adam indique que SGD avec momentum ($\beta = 0.9$) est le meilleur compromis pour ce problème, grâce à une convergence plus stable et des performances test supérieures.

Q6.2 – Analyse via matrices de confusion
Les matrices de confusion normalisées (Section 5) montrent que DeepCNN améliore nettement les classes difficiles (oiseaux, grenouilles, bateaux) par rapport à SimpleCNN, tout en conservant des erreurs typiques sur les paires sémantiquement proches (chat/chien, automobile/camion).

Q7 & Q8.2–Q8.3 – Cartes d'activation et évolution temporelle
Les visualisations d'activation (Section 6) confirment l'apprentissage hiérarchique des features (bords $\rightarrow$ textures $\rightarrow$ objets). L'évolution temporelle (Section 7) montre le passage d'activations bruitées (époque 1) à des patterns stables et structurés (époques 15–20), ce qui valide expérimentalement le processus d'apprentissage théorique d'un CNN.

8.4 Perspectives Futures

- Data augmentation (rotations, translations, flips) pour améliorer la robustesse
- Batch normalization pour stabiliser l'entraînement et accélérer la convergence
- Learning rate scheduling adaptatif (cosine, step decay)
- Transfer learning depuis des modèles pré-entraînés (ResNet, DenseNet)
- Exploration d'architectures plus récentes (Vision Transformers) pour comparaison

9. Annexe : Analyse détaillée des expériences PyTorch

Cette annexe reprend de manière quasi exhaustive le rapport d'analyse détaillée initialement rédigé à partir des expériences PyTorch (division des données, hyperparamètres, étude du surapprentissage, visualisation des activations, etc.). Elle permet de conserver l'intégralité du contenu scientifique de la première version du rapport tout en l'articulant avec les résultats plus récents obtenus avec Keras/TensorFlow dans le corps principal de ce document.

```
================================================================================
TP3 - RÉSEAUX DE NEURONES CONVOLUTIFS POUR LA CLASSIFICATION CIFAR10
ANALYSE COMPLÈTE ET RÉSULTATS
================================================================================

ENVIRONNEMENT ET CONFIGURATION
- Framework: PyTorch 2.11.0 (choisi pour compatibilité Python 3.14.2)
- Dataset: CIFAR10 (60,000 images 32×32, 10 classes)
- Device: Apple Metal Performance Shaders (mps)
- Taille des images: 32×32×3 pixels

================================================================================
PARTIE I: STRUCTURATION ET PRÉPARATION DES DONNÉES
================================================================================

1.1 DIVISION DES DONNÉES
---------------------------

Base d'entraînement: 50,000 images → Division en:
  • Ensemble d'entraînement: 5,000 images (10% du total)
  • Ensemble de validation: 1,000 images (2% du total)
  • Ensemble de test: 1,000 images (2% du total)

Raison de cette division:
  → Entraînement: Plus petit pour accélérer les expériences
  → Validation: Permet l'évaluation régulière pendant l'entraînement
  → Test: Évaluation finale sur données non vues
  → Les proportions (80/10/10) sont respectées relative aux 5,000 choisis

1.2 STANDARDISATION DES DONNÉES
---------------------------

Formule appliquée: X_std = (X - mean) / (std + 1e-7)

Statistiques calculées par canal (R, G, B):
  • Moyenne de chaque canal: calculée sur tout l'ensemble d'entraînement
  • Écart-type de chaque canal: calculé sur tout l'ensemble d'entraînement
  • Application: Pendant l'entraînement et le test

Impact:
  → Réduction du décalage intérieur (internal covariate shift)
  → Convergence plus rapide du réseau
  → Amélioration de la généralisation

1.3 FORMAT DES DONNÉES
---------------------------

Après prétraitement:
  • Shape entraînement: (5,000, 3, 32, 32) - Format PyTorch: NCHW
  • Shape validation: (1,000, 3, 32, 32)
  • Shape test: (1,000, 3, 32, 32)
  • Type de données: float32
  • Intervalle de valeurs: [-3, 3] après standardisation
```

==========
PARTIE II: ARCHITECTURE CNN ET PARAMÈTRES ENTRAÎNABLES
==========

2.1 ARCHITECTURE SIMPLE (SimpleCNN)

Couches:
1. Conv2D(3→8, kernel=3×3, padding=1, activation=ReLU)
   → Sortie: (batch, 8, 32, 32)
   → Paramètres: 3×3×3×8 + 8 = 224 paramètres

2. MaxPool2D(kernel=2×2, stride=2)
   → Sortie: (batch, 8, 16, 16)
   → Paramètres: 0

3. Flatten()
   → Sortie: (batch, 2048)
   → Calcul: 8 × 16 × 16 = 2048

4. Dense(2048→64, activation=ReLU)
   → Paramètres: 2048×64 + 64 = 131,136

5. Dense(64→10, softmax)
   → Paramètres: 64×10 + 10 = 650

TOTAL DE PARAMÈTRES ENTRAÎNABLES: 132,010

Justification:
→ 1 couche convolutive suffit pour détecter les features bas-niveau (arêtes, textures)
→ MaxPool réduit la dimensionnalité et contrôle le surapprentissage
→ 2 couches denses permettent la classification avec 64 neurones cachés

2.2 ARCHITECTURE PROFONDE (DeepCNN)

Couches:
1. Conv2D(3→16, kernel=3×3, padding=1, activation=ReLU)
   → Sortie: (batch, 16, 32, 32)
2. MaxPool2D(kernel=2×2)
   → Sortie: (batch, 16, 16, 16)
3. Conv2D(16→32, kernel=3×3, padding=1, activation=ReLU)
   → Sortie: (batch, 32, 16, 16)
4. MaxPool2D(kernel=2×2)
   → Sortie: (batch, 32, 8, 8)
5. Flatten() → 2,048 éléments
6. Dense(2048→128, Dropout=0.3, activation=ReLU)
7. Dense(128→10, softmax)

TOTAL DE PARAMÈTRES: ~265,000

Améliorations:
→ Plus de filtres (16→32) pour capturer des features complexes
→ 2 niveaux de pooling pour réduction progressive
→ Dropout(0.3) pour réguler le surapprentissage
→ Architecture hiérarchique pour représentations multi-niveaux

==========
PARTIE III: EXPÉRIMENTATION DES HYPERPARAMÈTRES
==========

3.1 EFFET DE LA TAILLE DE BATCH

Configuration:
- Époque: 20 itérations complètes sur l'ensemble d'entraînement
- Optimiseur: SGD avec momentum=0.9
- Taux d'apprentissage: 0.01
- Fonction de perte: CrossEntropyLoss

Résultats:

Batch Size = 16:
- Accuracy test: 42.50%
- Caractéristique: Plus de mises à jour de poids par époque
- Avantage: Convergence plus rapide, meilleure généralisation
- Inconvénient: Plus coûteux en temps de calcul

Batch Size = 32:
- Accuracy test: 34.10%
- Point de départ standard

Batch Size = 64:
- Accuracy test: 40.10%
- Équilibre entre vitesse et gradient

Batch Size = 128:
- Accuracy test: 36.40%
- Moins de mises à jour par époque
- Risque: Moins de signal de gradient, moins de régularisation naturelle

Observation principale:
→ Batch size = 16 offre le meilleur résultat (42.50%)
→ Contrairement à la sagesse classique, les petits batches donnent ici de meilleurs résultats
→ Cela peut être dû au bruit bénéfique qui agit comme une régularisation

3.2 COMPARAISON DES OPTIMISEURS

Configuration (fixe):
- Batch size: 32
- Époque: 20
- Taux d'apprentissage: 0.01

Résultats:

SGD (Stochastic Gradient Descent):
- Accuracy test: 33.50%
- Formule: $w_{t+1} = w_t - lr \times \nabla J(w_t)$
- Convergence: Basique, gradient brut

SGD + Momentum (β=0.9):
- Accuracy test: 41.70% ■ MEILLEUR
- Formule: $m_t = \beta \times m_{t-1} + \nabla J(w_t)$
  $w_{t+1} = w_t - lr \times m_t$
- Avantage: Accumulation de gradients passés, améliore convergence
- Direction: Plus lisse et accélérée

Adam (Adaptive Moment Estimation):
- Accuracy test: 37.00%
- Formule: Combine momentum et taux d'apprentissage adaptatif par paramètre
- Performance: Intermédiaire
- Raison possible: Hyperparamètres par défaut (β_1=0.9, β_2=0.999) non optimisés pour CIFAR10

Conclusion:
→ SGD avec momentum est le meilleur choix pour ce problème
→ Adam n'offre pas d'améliorations ici (nécessiterait tunning)

==========
PARTIE IV: ANALYSE DU SURAPPRENTISSAGE (OVERFITTING)
==========

4.1 COMPARAISON SimpleCNN vs DeepCNN

Configurération:
- Époque: 30 itérations
- Batch size: 32
- Optimiseur: SGD + momentum

Résultats finaux:

SimpleCNN:
- Architecture: 1 conv + Dense(64)
- Test Accuracy: 34.10%
- Nombre de paramètres: 132,010

DeepCNN (2 conv layers):
- Architecture: 2 conv + Dense(128) + Dropout(0.3)
- Test Accuracy: 53.80% ■ +19.7% d'amélioration
- Nombre de paramètres: 265,000
- Regularisation: Dropout(0.3) lors de l'entraînement

Analyse du surapprentissage:

Écart train-test observé:
- SimpleCNN: Train ≈ 76%, Test = 34% → Gap: 42% (surapprentissage sévère)
- DeepCNN: Train ~ 70%, Test = 53.8% → Gap: ~16% (contrôlé)

Facteurs de réduction du surapprentissage dans DeepCNN:
1. Architecture plus profonde: capture de patterns complexes
2. Dropout(0.3): désactivation aléatoire de 30% des neurones
3. Meilleure représentation de features par 2 niveaux de convolution
4. Pooling multiple: réduction progressive et invariance locale

4.2 RÔLE DU DROPOUT

Dropout(p=0.3):
- Probabilité de désactivation: 30% des neurones
- Effet d'ensemble virtuel: Chaque forward pass utilise un sous-réseau différent
- Mécanisme: Prévient la co-adaptation des neurones
- Résultat: Généralisation améliorée

Impact mesurable:
- Sans dropout: Surapprentissage direct (gap train-test large)
- Avec dropout(0.3): Réduction significative du surapprentissage

==========
PARTIE V: AMÉLIORATIONS ET TECHNIQUES DE RÉGULARISATION
==========

5.1 TECHNIQUES APPLIQUÉES
---

1. Augmentation d'architecture:
   ✓ Augmentation progressive des filtres (8→16→32)
   ✓ Ajout d'une deuxième couche convolutive
   ✓ Augmentation de la couche Dense (64→128)

2. Régularisation par Dropout:

✓ Dropout(p=0.3) et fonction de perte Dense
✓ Réduit la co-activation des neurones

3. Normalisation des données:
✓ Standardisation (mean=0, std=1) appliquée
✓ Stabilise l'apprentissage et accélère la convergence

4. Choix d'optimiseur:
✓ SGD + Momentum (β=0.9)
✓ Favorise une convergence régulière

5.2 RÉSULTATS DE L'AMÉLIORATION

Avant optimisation (SimpleCNN de base):
- Test Accuracy: ~40% (moyenne sur plusieurs runs)
- Surapprentissage: Sévère (train ~76%, test ~40%)

Après optimisation (DeepCNN + Dropout + Tunning):
- Test Accuracy: 53.80%
- Surapprentissage: Réduit (train ~70%, test ~53.8%)
- Amélioration: +13.8 points de précision

Chiffres clés de l'amélioration:
- Gap train-test: 42% → 16% (62% de réduction)
- Précision absolue: +13.8% (relative: +35%)

PARTIE VI: VISUALISATION DES CARTES D'ACTIVATION

6.1 OBJECTIF

Les cartes d'activation (activation maps) permettent de visualiser:
- Comment différents filtres convolutifs réagissent aux images
- Quelles features détecte chaque filtre
- Le fonctionnement interne du réseau de neurones

6.2 FICHIERS GÉNÉRÉS

1. activation_maps.png:
   - Affiche une image originale et ses 8 cartes d'activation brutes
   - 6 images d'exemple visualisées
   - Les filtres détectent des features différentes:
     * Filtre 1: Bordures/Edges horizontales
     * Filtre 2: Textures fines
     * Filtre 3: Points saillants
     * etc.

2. activation_heatmaps.png:
   - Affiche 3 colonnes par image:
     * Image originale (couleur)
     * Heatmap d'activation agrégée (colormap 'jet')
     * Superposition image + heatmap
   - Zones chaudes (rouges) = forte activation
   - Zones froides (bleues) = faible activation

3. activation_statistics.png:
   - Graphique 1: Activation moyenne par filtre
     * Montre quels filtres sont les plus actifs
     * Détecte les filtres "morts" (peu d'activation)

- Graphique 2: Distribution globale des activations
  * Vérifie la distribution des réponses du réseau

6.3 INTERPRÉTATION DES RÉSULTATS

Observations principales:

✓ Filtres spécialisés:
- Les 8 filtres présentent des réponses spécialisées
- Certains réagissent aux formes, d'autres aux textures
- Cela démontre l'apprentissage hiérarchique du réseau

✓ Activations locales:
- Les heatmaps montrent des activations concentrées sur des régions pertinentes
- Exemple: Pour les images de chats, activation autour de la tête et des oreilles
- Le réseau a appris à se concentrer sur les régions discriminantes

✓ Distribution des activations:
- Moyenne positive indique une bonne utilisation des ReLU
- Distribution proche de la normale (Gaussienne)
- Aucun filtre "mort" observé

PARTIE VII: RÉSUMÉ DES CONCLUSIONS

7.1 RÉPONSES AUX QUESTIONS CLÉS

Q1: Comment diviser les données de CIFAR10?
R: Division en 80% (5000→4000) / 10% (1000) / 10% (1000) pour train/val/test
  Standardisation imperactive pour convergence rapide et généralisatio

Q2: Pourquoi utiliser la standardisation?
R: Réduit le covariate shift interne, accélère convergence de 2-3x, améliore généralisation

Q3: Combien de paramètres entraînables dans SimpleCNN?
R: 132,010 = (224 conv) + (131,136 dense1) + (650 dense2)

Q4: Quel est le meilleur batch size?
R: 16 (42.50% accuracy) > 32 (34.10%) > 64 (40.10%) > 128 (36.40%)
  Les petits batches offrent une régularisation naturelle bénéfique

Q5: Quel optimiseur est préférable?
R: SGD + Momentum (41.70%) > Adam (37.00%) > SGD simple (33.50%)

Q6: Comment réduire le surapprentissage?
R: Dropout(0.3) + Architecture plus profonde + Regularisation
  Résultat: Gap train-test réduit de 42% à 16%

Q7: Comment exploiter les cartes d'activation?
R: Visualiser les features détectées par chaque filtre convolutif
  Vérifier que le réseau se concentre sur des régions pertinentes

7.2 AMÉLIORATIONS POTENTIELLES FUTURES

1. Data augmentation:
  - Rotations, translations, zoom des images d'entraînement
  - Attendu: +5-10% accuracy

2. Batch Normalization:

- Normalisation interne des activations
- Attendu: Convergence 2-3x plus rapide

3. Learning Rate Scheduling:
- Réduction du taux d'apprentissage au cours du temps
- Attendu: Meilleure convergence fine-tuning finale

4. Ensemble methods:
- Entraînement de plusieurs modèles avec moyennage
- Attendu: +2-3% supplémentaires

7.3 RESSOURCES UTILISÉES

Code Python:
- tp3_solution.py: Implémentation de base (132 lignes)
- tp3_advanced.py: Expériences avancées (450+ lignes)
- tp3_visualizations.py: Visualisations d'activation (350+ lignes)

Visualisations générées:
- batch_size_analysis.png
- optimizer_comparison.png
- overfitting_analysis.png
- activation_maps.png
- activation_heatmaps.png
- activation_statistics.png

Données:
- CIFAR10: 60,000 images (10 classes, 32×32 pixels)
- Ensemble d'entraînement: 5,000 images
- Ensemble de validation: 1,000 images
- Ensemble de test: 1,000 images

==========
CONCLUSION GÉNÉRALE
==========

Le TP3 démontre avec succès:

■ Implémentation d'une architecture CNN pour classification d'images
■ Compréhension des hyperparamètres (batch size, optimiseur, taux apprentissage)
■ Techniques de régularisation (dropout, architecture profonde)
■ Analyse quantitative dusurapprentissage et mitigation
■ Visualisation et interprétation des représentations internes du réseau
■ Amélioration de 35% de la précision grâce à l'optimisation

Architecture finale:
- SimpleCNN de base: 34-42% accuracy (selon les paramètres)
- DeepCNN optimisée: 53.80% accuracy
- Généralisation: Bon contrôle du surapprentissage

Le projet démontre les principes fondamentaux du deep learning moderne
et l'importance de l'expérimentation systématique en machine learning.

==========

Note: Ce rapport présente l'analyse complète du TP3 incluant tous les résultats requis (Q3.1-Q8.3). Tous les scripts utilisés pour générer ces résultats sont disponibles et reproductibles avec Keras/TensorFlow.

Auteurs: Lucca de Souza Amodio, João Victor Ferro Amorosini, Vinicius
Date: 24/03/2026